Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «Конструирование программного обеспечения»
Тема: «Создание ключевых функций»

Выполнил:
Студент 2 курса
250966
Группы ПО-17
Фещук А.А.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: реализовать основную логику игры на основе классов Mayor, Town и Building с недельным планированием развития города.

Ход работы

Вариант 4

4. Игра представляет собой симулятор градоправителя, развивающего город. Реализовать классы Mayor, Town и Building. Игрок должен составлять план развития города на неделю, тратя деньги из бюджета на постройку зданий, проведение мероприятий и т.д., получая по окончанию недели доход, зависящий от количества жителей и построенных зданий. Реализовать разные здания, которые можно строить для получения различных бонусов.

Код программы:

```
#include <iostream>
#include <fstream>
#include <queue>
#include <vector>
#include <string>
#include <limits>
#include <algorithm>
using namespace std;

struct GameSettings {
    int citySize = 10, budget = 1000, population = 100;
};

class SettingsManager {
    GameSettings s;
    queue<int> qSize, qBudget, qPop;

    void load(const string& f, queue<int>& q, const vector<int>& def) {
        ifstream in(f);
        int v;
        if (in.good()) { while (in >> v) q.push(v); return; }
        ofstream out(f);
        for (int x : def) { out << x << "\n"; q.push(x); }
    }

    int next(queue<int>& q, int cur) {
        if (q.empty()) return cur;
        int first = q.front(), res = first;
        for (size_t i = 0; i < q.size(); ++i) {
            int v = q.front(); q.pop(); q.push(v);
            if (v == cur) { q.pop(); res = q.empty() ? first : q.front(); q.push(res);
break; }
        }
        return res;
    }

public:
    SettingsManager() {
        load("city_size_values.txt", qSize, { 10, 20, 30 });
        load("budget_values.txt", qBudget, { 1000, 5000, 10000 });
        load("population_values.txt", qPop, { 100, 500, 1000 });
        ifstream in("settings.txt");
        if (!(in >> s.citySize >> s.budget >> s.population)) save();
    }
    GameSettings& get() { return s; }
    void save() {
        ofstream out("settings.txt");
        out << s.citySize << "\n" << s.budget << "\n" << s.population << "\n";
    }
    void nextSize() { s.citySize = next(qSize, s.citySize);   save(); }
```

```
    void nextBudget() { s.budget = next(qBudget, s.budget);     save(); }
    void nextPop() { s.population = next(qPop, s.population); save(); }
};

// ==================== КЛАССЫ ИГРЫ ====================
class Building {
public:
    string name;
    int cost, income, popBonus, happyBonus;
    Building(string n, int c, int inc, int pop, int hap)
        : name(n), cost(c), income(inc), popBonus(pop), happyBonus(hap) {
    }

    void printInfo(int idx) const {
        cout << " " << idx << ". " << name
            << " | цена: " << cost
            << " | доход/нед: " << income
            << " | жители: +" << popBonus
            << " | счастье: " << (happyBonus >= 0 ? "+" : "") << happyBonus << "\n";
    }
};

class Town {
public:
    int size, budget, population, happiness = 50;
    vector<Building> buildings;
    Town(int sz, int bud, int pop) : size(sz), budget(bud), population(pop) {}

    bool build(const Building& b) {
        if (budget < b.cost) {
            cout << "Недостаточно средств для постройки " << b.name << "!\n";
            return false;
        }
        if ((int)buildings.size() >= size) {
            cout << "Достигнут лимит зданий (" << size << ").\n";
            return false;
        }
        budget -= b.cost;
        population += b.popBonus;
        happiness = min(100, max(0, happiness + b.happyBonus));
        buildings.push_back(b);
        cout << "Построено: " << b.name << "\n";
        return true;
    }

    void status() const {
        cout << "--- СОСТОЯНИЕ ГОРОДА ---\n";
        cout << "Размер: " << size << " (зданий: " << buildings.size() << "/" << size
<< ")\n";
        cout << "Бюджет: " << budget << "\n";
        cout << "Население: " << population << "\n";
        cout << "Счастье: " << happiness << "/100\n";
        if (!buildings.empty()) {
            cout << "Построенные здания:\n";
            for (size_t i = 0; i < buildings.size(); ++i)
                cout << "  " << (i + 1) << ". " << buildings[i].name << "\n";
        }
    }

    int endWeek() {
        int income = 0;
        for (auto& b : buildings) income += b.income;
        int taxes = population / 10;
        double mod = 0.5 + happiness / 100.0;
        int total = int((income + taxes) * mod);
        budget += total;
        if (happiness >= 60) population += population / 20;
        else if (happiness <= 30) population -= population / 25;
        return total;
    }
};
```

```
class Mayor {
public:
    int actions = 5, week = 0;
    void reset() { actions = 5; }
    bool has() const { return actions > 0; }
    void use() { --actions; }
};

// ==================== УТИЛИТЫ ====================
void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

void wait() {
    cout << "\nНажмите Enter, чтобы продолжить...";
    cin.get();
}

int readInt(const string& p) {
    int v;
    while (true) {
        cout << p;
        if (cin >> v) { cin.ignore(10000, '\n'); return v; }
        cin.clear(); cin.ignore(10000, '\n');
        cout << "Ошибка ввода. Введите число.\n";
    }
}

// ==================== ЛОГИКА ИГРЫ ====================
void startGame(const GameSettings& cfg) {
    clearScreen();
    cout << "=== НАЧАЛО ИГРЫ: «ГОРОД МЕЧТЫ» ===\n\n";
    cout << "Вы вступили в должность градоправителя!\n";
    cout << "Начальные параметры (из настроек):\n";
    cout << "  Размер города: " << cfg.citySize << "\n";
    cout << "  Бюджет: " << cfg.budget << "\n";
    cout << "  Население: " << cfg.population << "\n";

    Town town(cfg.citySize, cfg.budget, cfg.population);
    Mayor mayor;
    vector<Building> catalog = {
        {"Жилой дом", 200, 20, 30, 5},
        {"Магазин",   300, 60,  5, 3},
        {"Фабрика",   500, 120,10,-5},
        {"Парк",      150, 5,   0,15},
        {"Школа",     400, 30,  5,10},
        {"Театр",     350, 40,  0,18}
    };

    const int MAX_WEEKS = 6;
    while (mayor.week < MAX_WEEKS) {
        mayor.reset();
        mayor.week++;
        cout << "\n============================================\n";
        cout << " НЕДЕЛЯ " << mayor.week << " / " << MAX_WEEKS << "\n";
        cout << "============================================\n";

        while (mayor.has()) {
            clearScreen();
            town.status();
            cout << "\nДействий осталось: " << mayor.actions << "\n";
            cout << "1. Построить здание\n";
            cout << "2. Провести мероприятие (+10 счастья, -100 бюджета)\n";
            cout << "3. Пропустить оставшиеся действия\n";
            int c = readInt("Выберите действие: ");
```

```
            if (c == 1) {
                clearScreen();
                cout << "=== ДОСТУПНЫЕ ЗДАНИЯ ===\n";
                for (size_t i = 0; i < catalog.size(); ++i)
                    catalog[i].printInfo(i + 1);
                cout << " 0. Отмена\n";
                int b = readInt("Выберите здание: ");
                if (b >= 1 && b <= (int)catalog.size()) {
                    if (town.build(catalog[b - 1])) mayor.use();
                    wait();
                }
            }
            else if (c == 2) {
                if (town.budget >= 100) {
                    town.budget -= 100;
                    town.happiness = min(100, town.happiness + 10);
                    cout << "Мероприятие проведено! Счастье +10.\n";
                    mayor.use();
                }
                else cout << "Недостаточно бюджета для мероприятия.\n";
                wait();
            }
            else if (c == 3) {
                mayor.actions = 0;
            }
            else {
                cout << "Неверный пункт меню!\n";
                wait();
            }
        }

        int income = town.endWeek();
        cout << "\n=== ИТОГИ НЕДЕЛИ " << mayor.week << " ===\n";
        cout << "Доход за неделю: " << income << "\n";
        cout << "Текущий бюджет: " << town.budget << "\n";
        cout << "Население: " << town.population << "\n";
        cout << "Счастье: " << town.happiness << "/100\n";
        wait();
    }

    clearScreen();
    cout << "=== ИГРА ЗАВЕРШЕНА ===\n";
    town.status();
    int score = town.budget + town.population * 2 + town.happiness * 5;
    cout << "\nИтоговый счёт: " << score << "\n";
    cout << "(бюджет + население*2 + счастье*5)\n";
    wait();
}

// ==================== МЕНЮ ====================
void settingsMenu(SettingsManager& sm) {
    int c = 0;
    while (c != 4) {
        clearScreen();
        cout << "====================================\n";
        cout << " НАСТРОЙКИ\n";
        cout << "====================================\n";
        cout << "1. Размер города: " << sm.get().citySize << "\n";
        cout << "2. Начальный бюджет: " << sm.get().budget << "\n";
        cout << "3. Количество жителей: " << sm.get().population << "\n";
        cout << "4. Назад\n";
        cout << "====================================\n";
        c = readInt("Выберите пункт: ");
        if (c == 1) sm.nextSize();
        else if (c == 2) sm.nextBudget();
        else if (c == 3) sm.nextPop();
    }
}

int main() {
    setlocale(LC_ALL, "Russian");
```

```
    SettingsManager sm;
    int choice = 0;

    while (choice != 5) {
        clearScreen();
        cout << "====================================\n";
        cout << " ГОРОД МЕЧТЫ\n";
        cout << "====================================\n";
        cout << "1. Начать игру\n";
        cout << "2. Загрузить игру\n";
        cout << "3. Настройки\n";
        cout << "4. О программе\n";
        cout << "5. Выход\n";
        cout << "====================================\n";
        choice = readInt("Выберите пункт меню: ");

        if (choice == 1) startGame(sm.get());
        else if (choice == 2) {
            clearScreen();
            cout << "=== ЗАГРУЗКА ИГРЫ ===\n";
            cout << "Функция пока недоступна.\n";
            wait();
        }
        else if (choice == 3) settingsMenu(sm);
        else if (choice == 4) {
            clearScreen();
            cout << "=== О ПРОГРАММЕ ===\n";
            cout << "Игра: «Город мечты»\n";
            cout << "Версия: 2.0 (ЛР3, вариант 4)\n";
            cout << "Студент: Фещук Артём\n";
            wait();
        }
        else if (choice == 5) cout << "Выход из программы...\n";
        else { cout << "Неверный пункт меню!\n"; wait(); }
    }
    return 0;
}
```

Вывод: разработана игровая логика симулятора градоправителя с постройкой зданий, расходованием бюджета и начислением дохода, а настройки из предыдущей лабораторной работы интегрированы в запуск игры.

Контрольные вопросы:

0. Алгоритм работы программы.
При запуске создаётся SettingsManager, который загружает значения размера города, бюджета и населения из файлов (при их отсутствии — создаёт с дефолтными наборами {10,20,30}, {1000,5000,10000}, {100,500,1000}). Главное меню позволяет начать игру, загрузить (заглушка), изменить настройки (переключение по кругу через queue) или выйти. При старте игры создаётся Town с параметрами из настроек и Mayor. Игра идёт 6 недель; в каждой неделе мэру даётся 5 действий: построить здание из каталога (списывается бюджет, добавляются жители/счастье, проверяется лимит зданий), провести мероприятие (+10 счастья, −100 бюджета) или пропустить. В конце недели endWeek() считает доход (доход зданий + налоги population/10, умноженные на модификатор счастья 0.5 + happiness/100), начисляет его в бюджет и корректирует население. По завершении выводится итоговый счёт бюджет + население*2 + счастье*5.

1. Классы, объекты, поля, методы.

- **Классы:** GameSettings (структура настроек), SettingsManager, Building, Town, Mayor.
- **Объекты:** sm (SettingsManager), town (Town), mayor (Mayor), элементы catalog (Building), s (GameSettings).
- **Поля:** citySize, budget, population (GameSettings); qSize, qBudget, qPop (очереди значений); name, cost, income, popBonus, happyBonus (Building); size, budget, population, happiness, buildings (Town); actions, week (Mayor).
- **Методы:** load, next, save, nextSize, nextBudget, nextPop, get (SettingsManager); printInfo (Building); build, status, endWeek (Town); reset, has, use (Mayor); свободные функции startGame, settingsMenu, readInt, clearScreen, wait, main.

2. Основные принципы ООП.

- **Инкапсуляция** — данные и логика объединены в классах; например, SettingsManager скрывает очереди и настройки, предоставляя методы nextSize/save/get.
- **Абстракция** — классы моделируют реальные сущности (здание, город, мэр), скрывая детали реализации за интерфейсом (town.build(...), mayor.use()).
- **Наследование** — в явном виде не используется (все классы самостоятельны), хотя структурно программа допускает расширение иерархии зданий.
- **Полиморфизм** — также не реализован явно (нет виртуальных функций/перегрузки операторов), поведение задаётся конкретными классами.

3. Конструкторы, их виды, различия.

- **Конструктор по умолчанию** — SettingsManager() (загружает файлы), Mayor (поля инициализированы значениями по умолчанию).
- **Параметризованный конструктор** — Town(int sz, int bud, int pop), Building(string n, int c, int inc, int pop, int hap).
- **Различия:** конструктор по умолчанию не принимает аргументов и создаёт объект с типовыми значениями; параметризованный позволяет задать начальное состояние. Оба вызываются автоматически при создании объекта; конструктор копирования в коде не определён (используется неявный).

4. Отладка программ.
В коде предусмотрены: проверка корректности ввода через readInt (сброс потока cin.clear() и очистка буфера при ошибке), проверки бизнес-логики (build возвращает false при нехватке средств или превышении лимита зданий), информативные сообщения об ошибках, пошаговый вывод состояния города (status()) и итогов недели. Также используются wait() для пошагового контроля выполнения и clearScreen() для удобства восприятия.

5. Экстремальное программирование.
XP — методология гибкой разработки, включающая: парное программирование, частые короткие итерации, непрерывное тестирование, простой дизайн, рефакторинг, коллективное владение кодом, непрерывную интеграцию и «игру в планирование». В данной работе элементы XP видны частично: простой дизайн, небольшие функции-утилиты, пошаговая проверка результата; однако полноценные тесты, парная работа и непрерывная интеграция не применялись.